tier1_recall_recovery_e2e

tier1_recall_recovery_e2e.sh

Revision: 1 **Last modified:** 2026-06-10T11:40:00Z

Overview

`tests/emulator/tier1_recall_recovery_e2e.sh` is the Tier-1 (emulated-device, no live hardware) **FAILURE → operator-RECALL(forward-fix) → device-RECOVERY** OTA end-to-end harness. It boots a containerized Helix OTA control plane on podman and drives the complete recall/recovery round-trip against it, extending the happy-path `tier1_full_lifecycle_e2e.sh` with the failure, forward-fix recall, and recovery steps.

It is the podman/container sibling of the in-process Go integration test `server/internal/deviceemu/recall_recovery_test.go` (which proves the same flow over real HTTP against an in-process `httptest` server). The Go test is the fast gate; this script is the full containerized proof.

Prerequisites

- A running **podman** machine (on macOS, `podman machine start`).
- **Go** toolchain (cross-compiles the static `linux/arm64` binaries on the host).
- Host tools: `openssl` (≥3, with ed25519), `xxd`, `base64`, `curl`, `jq`, `python3`.
- The `containers/images/ota-device-emu` submodule image source present (§11.4.76 containers-submodule mandate).

Usage

```
bash tests/emulator/tier1_recall_recovery_e2e.sh
KEEP_UP=1 bash tests/emulator/tier1_recall_recovery_e2e.sh   # leave the pod up
```

Inputs (environment, all optional)

Var	Default	Meaning
`PODMAN`	`podman`	podman binary

Var	Default	Meaning
CP_IMAGE	ota-control-plane:dev	control-plane image tag
EMU_IMAGE	ota-device-emu:dev	base emulator image tag
ADMIN_USER	admin@helix.example	seeded admin login
ADMIN_PASS	generated test secret	seeded admin password
HELIX_PORT	8080	control-plane published port
KEEP_UP	(unset)	non-empty ⇒ skip teardown

Outputs / Side-effects

- Cross-compiles bin/ota-server + bin/ota-device-emu (static linux/arm64).
- Builds the control-plane + derived ota-device-emu podman images.
- Creates a podman pod helix-tier1-recall with the control plane + three emulator runs; removes it on exit unless KEEP_UP (trap cleanup EXIT, §11.4.14).
- Generates an **ephemeral** ed25519 keypair in a mktemp dir, never committed, rm -rf'd on exit (§11.4.10).
- Writes captured podman logs + emulator Outcome JSON + server telemetry + rollback history + a transcript under docs/qa/<run-id>-recall-recovery-container/ (§11.4.83).

Internal behaviour (the seven asserted steps)

1. **APPLY** — emulator (current 1.0.0) registers, gets a 200 offer of 1.1.0 carrying deployment D1, applies, advances, healthy, telemetry accepted (rejected=0).
2. **FAILURE** — the host re-registers the same hardware-id (idempotent via Idempotency-Key ⇒ a fresh device token + the SAME device id) and POSTs a real failure telemetry event (error_code=post_apply_health_check_failed) through the device-scoped /client/telemetry endpoint. The server marks the device unhealthy; the version does NOT advance. The emulator CLI does not expose a failure trigger, so the host drives it faithfully over the same device protocol — not a server-internal poke.
3. **server failure telemetry** — GET /devices/{id}/telemetry?event=failure shows the failure stamped with D1 + the error code.
4. **RECALL** — operator stages a forward-fix release 1.2.0 (release only; the recall endpoint creates the new deployment) and POST /deployments/{D1}/recall with to_release_id = the 1.2.0 release. Asserts 201 + kind=rollback + from=1.1.0 release + to=1.2.0 release + details.mode=forward-fix + a non-empty recall_deployment_id.
5. **rollback_history** — GET /deployments/{D1}/rollbacks confirms the forward-fix row.
6. **RECOVERY** — emulator (current 1.1.0) re-checks, gets a 200 offer of 1.2.0 carrying the recall deployment id, applies, advances to 1.2.0, healthy again, telemetry accepted; server success telemetry is stamped with the recall deployment id; server device status shows health.ok=true and current_version=1.2.0.
7. **ON-TARGET** — a third emulator run (current 1.2.0) returns 204 on-target.

Edge cases

- **Anti-downgrade invariant.** `handleClientUpdate` only offers a version strictly greater than the device's current. The device fails AT `1.1.0`, so the forward-fix MUST be HIGHER (`1.2.0`). Staging the fix below `1.1.0` would (correctly) yield a 204 and no recovery — that is why the script uses `1.2.0`.
- **409 on a second deployment.** The control plane returns 409 if a second all-targets deployment is created while D1 is still active. The script therefore stages the fix as a **release only** and lets the recall endpoint create the recall deployment.
- **Stale-listener guard.** The script fails fast if port `HELIX_PORT` is already serving `/healthz` (a stale control plane), to avoid a misleading downstream FAIL.

Related scripts

- `tests/emulator/tier1_full_lifecycle_e2e.sh` — the happy-path full-lifecycle sibling this script extends.
- `tests/e2e/pipeline_signed.sh` — the artifact signing contract mirrored here.
- `server/internal/deviceemu/recall_recovery_test.go` — the in-process Go integration test proving the same flow.
- `docs/scripts/tier1_full_lifecycle_e2e.md` — the sibling companion doc.

Last verified

- **2026-06-10** — `bash -n` and `sh -n` clean (§11.4.67). The in-process Go proof (`go test -run RecallRecovery ./internal/deviceemu/...` and `go test -race ...`) PASSED. The live podman run is to be executed by the conductor (the parallel phase did not run podman to avoid contending with a concurrent agent).